

<!-- ! promise -->

Definition:

- A ``Promise`` in JavaScript is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

States of a Promise:

1. **Pending**: Initial state, neither fulfilled nor rejected.
2. **Fulfilled**: The operation was completed successfully.
3. **Rejected**: The operation failed.

Basic Syntax:

```
let promise = new Promise(function(resolve, reject) {  
    // Asynchronous operation  
    if (/* operation successful */) {  
        resolve("Success!");  
    } else {  
        reject("Error!");  
    }  
});
```

Using ``then``, ``catch``, and ``finally``:

- ``then(onFulfilled, onRejected)``: Executes when the promise is fulfilled or rejected.
- ``catch(onRejected)``: Executes when the promise is rejected (a shorthand for ``then(null, onRejected)``).
- ``finally(onFinally)``: Executes when the promise is settled (either fulfilled or rejected), allowing for cleanup operations.

<!--! Static Methods of Promise -->

1. ``Promise.any(iterable)``

- it will show the first resolved promise

2. ``Promise.all(iterable)``

- If all the promises are resolved then it will show all the resolved promises
- if any of the promises is rejected then it will show the rejected promise .

3. `Promise.allSettled(iterable)`

- It will give array of objects .
- If any promise is rejected then it will show the status as rejected and give the reason of reject .

4. `Promise.race(iterable)`

- Returns a promise that resolves or rejects as soon as one of the promises in the iterable resolves or rejects, with the value or reason from that promise.

```
// ! how to create promise and handle the resolve and reject

let p = new Promise((resolve,reject)=>{

    resolve("hello everyone")
    reject("hiiiiiiiiii")
})

p.then((data)=>{
    console.log(data)
}).catch((err)=>{
    console.log(err)
})

console.log('-----')

let p1 = new Promise((resolve,reject)=>{

    resolve("resolve 1")
    reject("reject 1")
})

let p2 = new Promise((resolve,reject)=>{

    // resolve("resolve 2")
    reject("reject 2")
})
```

```
let p3 = new Promise((resolve, reject) => {
    resolve("resolve 3")
    reject("reject 3")
})

let p4 = new Promise((resolve, reject) => {
    resolve("resolve 4")
    reject("reject 4")
})

// ! Promise.any()

Promise.any([p4, p2, p3, p1])
    .then((data) => {
        console.log(data)
    })
    .catch((err) => {
        console.log(err)
    })

// ! Promise.all()

Promise.all([p4, p2, p3, p1])
    .then((data) => {
        console.log(data)
    })
    .catch((err) => {
        console.log(err)
    })

// ! Promise.allSettled

Promise.allSettled([p4, p2, p3, p1])
    .then((data) => {
        console.log(data)
    })
    .catch((err) => {
        console.log(err)
    })
```

```
// ! Promise.race()

Promise.race([p2,p3,p1,p4])
  .then((data)=>{
    console.log(data)
  })
  .catch((err)=>{
    console.log(err)
  })
```